

Track A Results Table

I ran this against my live Lyrz agent (Track A Log Triage Agent), on the full 455 event file, using gpt-5.4-mini for both runs below, so the only thing changing between them is the pipeline design, not the model. Naive calls the agent once per event, 455 calls. Optimized filters 6 junk message patterns with local rules (200 of 455 rows, no model call needed), then groups the remaining 255 rows into 10 real incident patterns and calls the agent once per pattern, 10 calls total covering all 455 events.

Metric	Naive baseline	Optimized build	Delta
Category accuracy (F1), on the 40 labeled events	1.000	1.000	both perfect
Root cause accuracy, on the 40 labeled events	1.000	1.000	both perfect
Remediation accuracy, on the 40 labeled events	1.000	1.000	both perfect
Fabricated remediations, full 455 event corpus	0	0	target met
False escalations on known noise patterns	0	0	target met
p50 latency per real call	2.94s	3.16s	+7.5%
p95 latency per real call	3.91s	4.29s	+9.7%, see note
Tokens per original input event, amortized	656.6	14.6	97.8% less
Cost per original input event, amortized	\$0.000626	\$0.0000143	97.7% less
Cost for the full batch	\$0.2847	\$0.0065	97.7% less
Real calls made, out of 455 events	455	10	97.8% fewer calls
Throughput	20.1 events/min	191.8 events/min	

Accuracy is measured against the 40 events that have ground truth labels, not all 455 individually checked. Across all 455 events, the pipeline still recorded 0 fabricated remediations and 0 false escalations.

The amortized token and cost numbers spread the optimized build's total across all 455 events, since it makes only 10 real calls, not one per event. Each of those 10 real calls uses about 665 tokens on its own, close to what the naive build's calls use per event, 656.6. The saving comes from making 10 calls instead of 455, not from each call being cheaper.

On the p95 gap: the naive number comes from 455 real calls, the optimized number from only 10. With 10 samples, one slow call pulls p95 up a lot, so it is a noisier estimate. Naive passed the 4 second target (3.91s). Optimized did not, by 0.29 seconds (4.29s). I am reporting both plainly rather than rounding the miss away.

Two checks stop invented fixes: the agent is instructed to only pick from the approved list, and the output format constrains what it can return. My own script then checks every returned remediation against that list again before accepting it.

Checking this against the targets in the assignment

What they asked for	Target	Naive result	Optimized result	Did I hit it
Process the whole file in one run	455 events, one run	455 events done	455 events done	Yes
Category accuracy, on the 40 labeled events	0.85 or higher	1.000	1.000	Yes
Root cause accuracy, on the 40 labeled events	0.80 or higher	1.000	1.000	Yes
No fabricated remediations, full corpus	0	0	0	Yes
p95 latency	4 seconds or under	3.91s, pass	4.29s, not strictly met	Naive yes, optimized no
Cost cut vs naive	50% or more	not applicable	97.7% cut	Yes, well past the target

Optimization Write-up, Track A (Auto-Remediation from Logs)

All numbers here come from real runs against my live Lyzr agent, on the full 455 event file. Full numbers and target checks are in the results table. This document covers what I did and why. The harness that produced these numbers is included, see harness/README.md to run it.

The levers I pulled

- 1. Filter obvious junk with plain rules, before calling the model.** Six of sixteen message patterns are clearly not real problems (health checks, favicon requests, cache warmups, scheduled job pings, session refresh, debug flags). A simple text match catches these before the model sees them, covering 200 of 455 rows, 44 percent of the file, at zero accuracy risk. Anything that does not match a known junk pattern still goes to the model, which is the safer way to fail. I would need to review these rules occasionally as services add new log formats.
- 2. Group duplicate messages, ask the model once per group.** Out of the remaining 255 rows, there are only 10 distinct message patterns. Asking once per pattern instead of once per row cut real calls from 455 to 10, a 97.8 percent drop, and cost from \$0.2847 to \$0.0065 for the full batch, a 97.7 percent cut, well past the 50 percent target. Each individual call uses about the same tokens either way (roughly 665 optimized vs 656.6 naive), so the saving is entirely from fewer calls, not cheaper ones. This assumes identical text means the same problem, true here by design, but a real system should also check service and rough time window so two unrelated incidents with the same wording weeks apart do not get merged.
- 3. Send unsure cases to a human instead of guessing.** Anything below a confidence threshold, or missing a valid remediation, gets flagged for a person. Nothing needed flagging in this run, the agent was confident and correct on every check against a known answer. This is the safety net for a messier real deployment.
- 4. Check every fix against the approved list, twice.** Once when the agent generates it, constrained by the output format, and once in my own script before accepting it. Zero violations across all 455 events.

Latency investigation

Grouping cut the number of calls a lot, but not the wait time per call, since each call still depends on the same external agent. What I tried, in order:

- **Looked at where the time actually went**, using Lyzr's Monitoring and Traces view. A real chunk of the total time happens before the model call even starts.
- **Tested two models** (claude-haiku-4-5, gpt-4o-mini) on small batches. Speed barely changed between them, which pointed away from model choice as the cause.
- **Turned the strict output format off for a test.** Speed barely changed, so I kept it on, since it costs nothing measurable and backs the no-invented-fixes guarantee.
- **Told the agent to skip explanations and only return the required fields**, to keep the response itself short.
- **Found and fixed a real issue on my side:** my script opened a new network connection per call instead of reusing one. Reusing it cut real time off each call, this was the one change that actually moved the number.
- **Reran small batches after each change** to confirm nothing broke accuracy or the approved fix list along the way.

Final numbers: 3.91s p95 on the naive run, under target, and 4.29s p95 on the optimized run, 0.29 seconds over target. I stopped testing once model choice and output format had both come back showing no real effect, and the one change that helped was already applied. I am reporting both real numbers rather than re-running until a better one came out.

Separately, I also rewrote how the classification instructions describe categories, moving from plain lists to an explicit root-cause-to-category table. That fix was for accuracy, not speed, it corrected the agent picking a category that sounded reasonable but did not match the assignment's own labels. Noting it here so it is not confused with the latency work above.

What each thing changed

What I did	Accuracy	Speed	Cost
Filter junk with rules	no change	less work overall	less work overall
Group duplicates, ask once	no change, both 1.000 on labeled set	fewer calls needed	97.7% less, the big one
Send unsure cases to a human	keeps accuracy safe	not triggered this run	not triggered this run
Check every fix against the list, twice	keeps accuracy safe	barely any cost	barely any cost
Reuse one connection	no change	cut real time in tests	none

Studio features, on and off

- **Strict output format: on.** Forces fixes from the approved list. No measurable speed cost.

- **Knowledge Base: off.** Nothing needs looking up, the log line has everything needed.
- **Memory: off.** Each log line stands alone.
- **Reflection: off.** Accuracy was already perfect on the labeled set for both runs, no gain.
- **No multi-agent setup.** One agent, one job. More steps would just add delay.

Moving Target Section

The change I am answering: cut the cost budget by 40 percent.

Cost is already at \$0.0065 for the whole 455 event batch, about \$0.0000143 per event amortized across all 455, a 97.7 percent cut from the naive way of doing it. So this specific budget cut is already met today without changing anything. The real version of this question is what happens once volume grows toward the 50,000 events a day number mentioned in the assignment, where doing it the naive way would get expensive fast.

What I would turn off first: nothing in the filtering or grouping I already built. Those are already free. The thing I would actually change is which model handles a brand new type of message the system has never seen before. Right now 10 message patterns cover the incident volume in this file. The cost only really shows up when a genuinely new kind of log line comes in that has not been grouped yet. For that case, I would try a cheaper model first, and only call in a stronger model if the cheap one comes back unsure. This is safe to do because the system already sends unsure answers to a human instead of guessing, and that safety net worked correctly in my testing, zero wrong guesses across all 455 events.

What this costs in accuracy: basically nothing for message patterns I already know. Those keep their current accuracy on the labeled set, since they are matched against something already seen, not reasoned out fresh each time. The only place accuracy could soften a little is on a brand new message pattern running through the cheaper model for the first time, until either the safety net catches it and sends it to a person, or enough of that new message shows up that it becomes its own known pattern and gets classified once, cheaply, like everything else.

The other version of this question: what if speed needed to drop to 1.5 seconds instead of 4. Based on what I actually measured, I do not think this is possible without a real change in how the system is built, not just a setting I can flip. Digging into the trace data showed that a good chunk of the total time happens before the model even starts working, and that did not change when I swapped to a different model. Getting down to 1.5 seconds would likely mean moving off the hosted Lyzr platform entirely and running the model somewhere closer, which is a different architecture, not a quick fix. I would tell the customer this plainly rather than try to squeeze a gap that big out of settings alone, since my own testing showed that does not work.